

AI Smart Bug Analyzer & Fix Advisor

Milestone 1 Documentation (30 June – 9 July)

Project Title: AI Smart Bug Analyzer & Fix Advisor

Team: Group 2

1. Objective

The objective of this milestone is to study the defect analysis process, understand Retrieval-Augmented Generation (RAG), analyze historical bug datasets, and build the initial Bug Submission Module that will later be integrated with an AI-based bug analysis and fix recommendation system.

2. Understanding the Problem Statement

Modern software projects contain thousands of historical bug reports. Developers often spend significant time searching for similar issues and identifying possible fixes.

The AI Smart Bug Analyzer aims to:

- Analyze incoming bug reports.
- Search similar historical bugs.
- Predict bug severity.
- Suggest possible fixes.
- Reduce debugging time using AI.

3. Study of Defect Analysis Workflow

The software defect lifecycle was studied to understand how bugs move through different stages.

Bug Identified
↓
Bug Submitted
↓
Bug Assigned
↓
Bug Analysis

↓
Fix Applied

↓
Testing

↓
Verification

↓
Closed

This workflow helped in understanding where the AI Smart Bug Analyzer fits into the software development process.

4. Study of RAG (Retrieval-Augmented Generation)

The RAG architecture was studied because it allows the system to retrieve relevant historical bug reports before generating fix suggestions.

Workflow:

User Bug Report

↓
Generate Embedding

↓
Vector Database Search

↓
Retrieve Similar Bugs

↓
LLM Generates Fix Suggestion

Benefits:

- Uses previous bug knowledge.
- Produces more accurate suggestions.
- Reduces hallucination.

5. Study of Semantic Similarity

Semantic similarity enables the system to compare the meanings of bug reports instead of relying only on exact keyword matching.

Example:

Bug A:

Application crashes during login.

Bug B:

Login page closes unexpectedly.

Meaning → Same

Words → Different

This concept is essential for retrieving relevant historical defects.

6. Dataset Collection

The Bugzilla historical bug dataset was collected and analyzed.

Project directory:

bugzilla bug reports/

corpus (fixsev).txt

embedding.npy

fix.csv

fix_train.csv

fix_test.csv

sev.csv

sev_train.csv

sev_test.csv

vocab.lst

7. Dataset Exploration

corpus (fixsev).txt

Contains raw historical bug reports.

Information observed:

- Bug descriptions

- Operating system
- Developer comments
- Additional details
- Resolution status
- Verification information

Purpose:

Used as the primary knowledge source for retrieval.

embedding.npy

Stores vector embeddings of historical bug reports.

Purpose:

Allows fast semantic similarity search.

fix.csv

Dataset used for bug fix prediction.

Contains bug descriptions and corresponding fix labels.

fix_train.csv

Training data used to train the bug fix prediction model.

fix_test.csv

Testing dataset used for model evaluation.

sev.csv

Dataset used for severity prediction.

sev_train.csv

Training data for severity classification.

sev_test.csv

Testing data for severity classification.

vocab.lst

Contains vocabulary generated from the corpus.

Used during preprocessing and vectorization.

8. Understanding Bug Report Structure

Each bug report generally contains:

- Title
- Description
- Steps to reproduce
- Environment
- Error logs
- Stack trace
- Expected result
- Actual result
- Resolution

Understanding this structure is necessary for AI-based analysis.

9. Bug Submission Module Development

A basic bug submission interface was developed using Python.

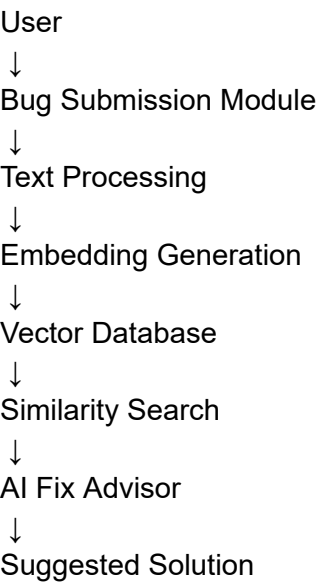
Features:

- Enter bug title
- Enter bug description
- Paste stack trace
- Upload bug report files

Purpose:

Collect structured bug reports for further AI processing.

10. System Architecture Designed



11. Technologies Studied

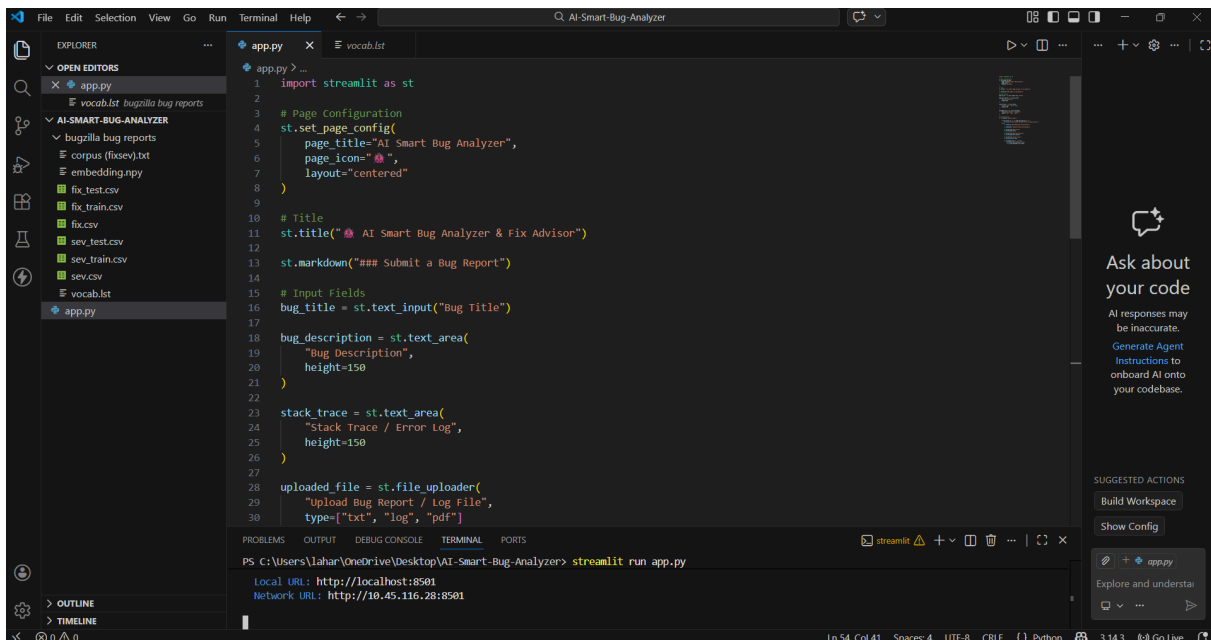
Technology	Purpose
Python	Backend development
Streamlit	User Interface
NumPy	Numerical processing
Bugzilla Dataset	Historical bugs
Embeddings	Semantic representation
RAG	Retrieval-based AI

Vector
Database

Similarity search

12. Outcomes Achieved

- Studied software defect workflow.
- Understood Retrieval-Augmented Generation.
- Learned semantic similarity concepts.
- Explored Bugzilla historical datasets.
- Understood the purpose of each dataset file.
- Designed the system architecture.
- Developed the Bug Submission Module.
- Prepared the dataset for future AI integration.



Deploy

 **AI Smart Bug Analyzer & Fix Advisor**

Submit a Bug Report

Bug Title

NullPointerException while logging into the application

Bug Description

Expected Result:

The application should display an error message asking the user to enter valid credentials.

Actual Result:

The application crashes with a NullPointerException.

Stack Trace / Error Log

Exception in thread "main" java.lang.NullPointerException
at com.example.login.LoginService.authenticate(LoginService.java:42)
at com.example.login.LoginController.login(LoginController.java:18)
at com.example.Main.main(Main.java:10)

Upload Bug Report / Log File

Choose a file to upload

Agent Responsibilities

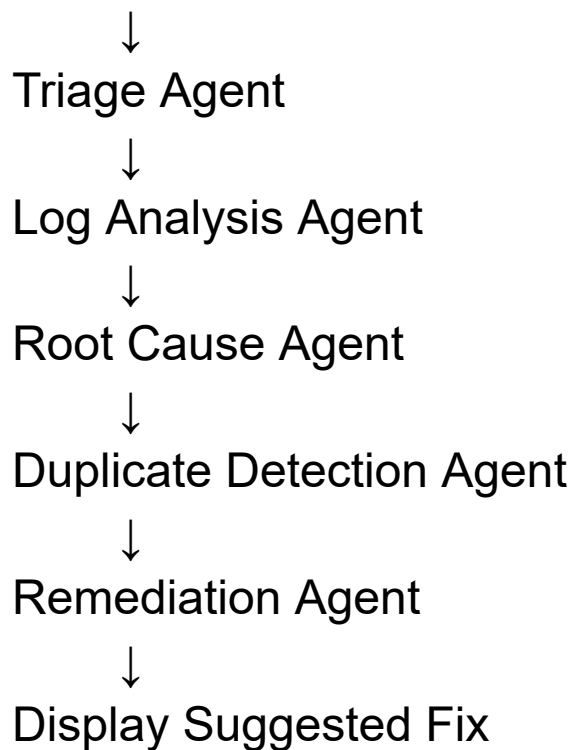
Agent	Responsibility
Triage Agent	Receives the bug report and categorizes it.
Log Analysis Agent	Analyzes stack traces and error logs.
Root Cause Agent	Identifies the possible root cause of the bug.
Duplicate Detection Agent	Searches for similar historical bug reports.
Remediation Agent	Suggests possible fixes using retrieved historical bugs.

Orchestration Flow

User submits bug report



Bug Submission Module



Knowledge Base Data Model

Field	Description
Bug ID	Unique bug identifier
Bug Title	Title of the bug
Description	Detailed bug description
Stack Trace	Error log or exception details
Severity	Bug severity level

Resolution Fix applied for the bug

Comments Developer discussion

Embedding Vector representation used
for similarity search

Dataset Preprocessing

The raw Bugzilla dataset was preprocessed to improve data quality before further processing in the RAG pipeline.

Steps Performed

- Loaded the `corpus (fixsev).txt` dataset.
- Removed unnecessary blank lines.
- Removed extra leading and trailing spaces.
- Preserved important information such as bug descriptions, stack traces, developer comments, and resolution details.
- Saved the cleaned dataset as `clean_bug_reports.txt` for the next stages of the project.

Output: A cleaned dataset ready for chunking, embedding generation, and similarity search.

(Important to do):

This is the complete pipeline

User uploads bug

↓

Preprocessing

↓

Chunking



Embedding Generation



FAISS Index



Similarity Search



Retrieved Bugs

Notice

LLM is NOT required yet.

LLM comes in Milestone 2 or later.

9. Chunking

Objective

To divide the cleaned Bugzilla dataset into smaller bug report chunks so that each chunk can be processed independently during embedding generation and similarity search.

Implementation

- Loaded the cleaned dataset (`clean_bug_reports.txt`).
- Used Python's **Regular Expression** (`re.split()`) to split the dataset based on the "**Created by**" keyword, which marks the beginning of each bug report.
- Removed any empty chunks generated during splitting.
- Stored all bug report chunks in a new file named `chunks.txt`.

Output

A file named **chunks.txt** containing individual bug report chunks, ready for embedding generation and vector database indexing.

10. Embedding Generation

Objective

To convert each bug report chunk into a numerical vector (embedding) that captures its semantic meaning. These embeddings will be used for similarity search in the RAG pipeline.

Implementation

- Loaded the chunked bug reports from **chunks.txt**.
- Used the **SentenceTransformer** model (**all-MiniLM-L6-v2**) to generate embeddings.
- Converted each bug report chunk into a **384-dimensional vector**.
- Saved all generated embeddings in **embeddings.npy** using NumPy.

Output

- **Input:** **chunks.txt**
- **Output:** **embeddings.npy**

Purpose

- Capture the semantic meaning of bug reports.

- Enable efficient similarity matching between new and historical bugs.
- Prepare the data for vector database indexing (FAISS).

11. Vector Database Indexing (FAISS)

Objective

To store the generated embeddings in a vector database for efficient similarity search.

Implementation

- Loaded the generated embeddings from `embeddings.npy`.
- Converted embeddings to `float32` format.
- Created a FAISS index using `IndexFlatL2`.
- Added all embeddings to the index.
- Saved the vector database as `bug_index.faiss`.

Output

- **Input:** `embeddings.npy`
- **Output:** `bug_index.faiss`

Purpose

- Store embeddings efficiently.
- Enable fast semantic similarity search.
- Prepare the historical defect knowledge base for retrieving similar bug reports.

12. Similarity Search

Objective

Retrieve the most semantically similar historical bug reports for a newly submitted bug.

Implementation

- Loaded the FAISS vector database (`bug_index.faiss`).
- Loaded the SentenceTransformer model (`all-MiniLM-L6-v2`).
- Converted the user's bug description into an embedding.
- Compared the query embedding with all stored embeddings using FAISS.
- Retrieved the **Top 5** most similar bug reports.

Output

- **Input:** User bug description.
- **Output:** Top 5 similar historical bug reports.

Purpose

- Identify duplicate or related bugs.
- Assist developers by retrieving similar historical issues.
- Improve bug analysis using semantic similarity.

GITHUB REPO LINK: <https://github.com/Lahari-333/AI-Smart-Bug-Analyzer-Fix-Advisor.git>

Dataset Source:

Bugzilla Historical Bug Reports
(Source: Kaggle)